

# Assignment 2 – Cloud Deployment Report

## (Google Dataproc)

This report documents the complete cloud deployment process undertaken for Assignment 2, which focused on executing a Hadoop MapReduce and SparkML workflow using Google Cloud Dataproc. The report mirrors the structure of the Assignment 1 deployment report while expanding to cover the additional complexity, trials, configuration errors, and resolutions encountered during Assignment 2.

### Cloud Platform Selection

Google Cloud Dataproc was selected as the cloud platform for this assignment. Dataproc provides a fully managed environment for Hadoop, Pig, Hive, and Apache Spark, with native integration to Google Cloud Storage and YARN for distributed resource management. The availability of free trial credits and strong support for Jupyter notebooks made Dataproc particularly suitable for interactive SparkML analysis.

### Initial Setup and IAM Configuration

A new Google Cloud project was created specifically for Assignment 2. During initial cluster creation attempts, multiple IAM validation errors were encountered. These errors were caused by missing permissions on the Compute Engine default service account. To resolve this, the following roles were explicitly assigned: Dataproc Worker, and Storage Admin. Once these roles were applied, Dataproc cluster creation proceeded successfully.

### Dataproc Cluster Deployment

A standard Dataproc cluster was deployed in the *europa-west1* region with Jupyter Notebook support enabled. This step was critical, as earlier cluster deployments without Jupyter support prevented notebook based execution. The cluster consisted of one master node and multiple worker nodes, providing sufficient resources for Pig and Spark workloads. The cluster status was verified as RUNNING before proceeding.

### Data Ingestion and HDFS Configuration

Input datasets were uploaded to Google Cloud Storage and subsequently ingested into HDFS using the *hdfs dfs -put* command. Dedicated HDFS directories were created under */user/hadoop/data* and */user/hadoop/outputs*. This ensured all downstream Pig and Spark jobs operated on distributed storage rather than local files which required some changing to our initial code.

### Pig Processing and MapReduce Execution

Apache Pig scripts were submitted to Dataproc as MapReduce jobs to perform data cleaning, grouping, and joining. Several execution failures were encountered due to memory pressure and existing HDFS output directories. These issues were resolved by explicitly removing output directories before re-running jobs. All Pig jobs eventually completed successfully, producing cleaned and joined datasets in HDFS.

## Spark and SparkML Execution via Jupyter Notebook

A PySpark Jupyter notebook was attached directly to the Dataproc cluster using the JupyterLab web interface. The Spark Session was managed automatically by Dataproc and executed on YARN, avoiding local execution mode. The notebook loaded data from HDFS, applied Spark transformations, and implemented SparkML components including feature vectorisation and similarity analysis. Issues appeared to arise in possible memory issues but were eventually resolved through trial and error.

## Monitoring and Validation

Execution was continuously monitored using the YARN ResourceManager web interface. Multiple Spark applications were observed in FINISHED and SUCCEEDED states, confirming successful distributed execution across the cluster. Pig MapReduce jobs were also visible in the YARN history, providing end-to-end validation of the workflow. It was essential to use this UI as I discovered now resource allocations were possible if a previous spark session was still running, hence so many were shown in the screencast as well as the odd failed one which was killed manually. After all screen recording and snapshots were taken, the cluster was deleted for efficiency purposes.

## Conclusion

This assignment successfully demonstrated the deployment and execution of a Hadoop MapReduce and SparkML pipeline using Google Cloud Dataproc. Despite multiple configuration and resource-related challenges, all issues were systematically resolved. The final solution mirrored the structure of Assignment 1 while extending it to support interactive Spark-based machine learning analysis in a cloud-native environment.

### LINK to Screencast:

[https://drive.google.com/drive/folders/1z2DI7snKkhI4MtorbaS71wsUg4v4QdH\\_?usp=drive\\_link](https://drive.google.com/drive/folders/1z2DI7snKkhI4MtorbaS71wsUg4v4QdH_?usp=drive_link)